



**POLITECNICO**  
MILANO 1863



**ITECNICO**  
LANO 1863

**architettura dei calcolatori e sistemi operativi**

**assembler**

**rappresentazione delle istruzioni**

**RISC V – capitolo 2 P&H**

# linguaggio macchina

le istruzioni in linguaggio assembler vanno tradotte in linguaggio macchina (cioè in sequenze di 0 e 1) per essere eseguite

le operazioni di un linguaggio macchina / assembler, incluso RISC V, riguardano:

- la manipolazione di dati (istruzioni aritmetiche/logiche)
- il trasferimento di dati (load / store)
- il controllo del flusso di istruzioni (salti)



# registri

- le istruzioni che movimentano i dati trasferiscono dati dalla memoria ai registri e viceversa
- in RISC V ci sono 32 registri generali
  - in assembler RISC V li possiamo nominare come:  
 $x0, x1, \dots, x31$
  - li possiamo anche nominare con i nomi simbolici ABI:
    - *zero*
    - *sp, ra, gp, fp, tp*
    - *s0, s1, s2, ..., s7, ...*
    - *t0, t1, t2, ...*
    - *a0, a1, a2, a3, ...*



# registri e linguaggio macchina

è necessaria una convenzione per rappresentare i registri tramite numeri in RISC V, l'indice nella notazione simbolica  $x0, x1, \dots, x31$  è il valore numerico che identifica il registro in linguaggio macchina:

| #       | nome ABI        | utilizzo                                                                                    |
|---------|-----------------|---------------------------------------------------------------------------------------------|
| 0       | <i>zero</i>     | costante 0 – imm modificabile (anche se viene scritta non cambia valore)                    |
| 1       | <i>ra</i>       | return address – indirizzo di rientro da funzione                                           |
| 2       | <i>sp</i>       | stack pointer – puntatore alla cima della pila                                              |
| 3       | <i>gp</i>       | global pointer – puntatore all'area dati globale QUI NON USATO                              |
| 4       | <i>tp</i>       | thread pointer – puntatore a thread QUI NON USATO                                           |
| 5 – 7   | <i>t0 – t2</i>  | valori temporanei (indirizzi, variabili globali e calcolo delle espressioni)                |
| 8       | <i>s0 / fp</i>  | variabile locale di funzione / frame pointer (se in uso)                                    |
| 9       | <i>s1</i>       | variabile locale di funzione                                                                |
| 10 – 11 | <i>a0 – a1</i>  | valore restituito da funzione (di solito basta <i>a0</i> – se serve si aggiunge <i>a1</i> ) |
| 12 – 17 | <i>a2 – a7</i>  | argomenti in ingresso a funzione (max 6 argomenti, gli eccedenti stanno in pila)            |
| 18 – 27 | <i>s2 – s11</i> | variabili locali di funzione (max 10 o 11 variabili, le eccedenti stanno in pila)           |
| 28 – 31 | <i>t3 – t6</i>  | valori temporanei (indirizzi, variabili globali e calcolo delle espressioni)                |



# formato delle istruzioni RISC V (RV64i) – i

tutte le istruzioni RISC V hanno la **stessa** dimensione (**32 bit**)

i 32 bit hanno un significato diverso a seconda del formato (o tipo) di istruzione

- il tipo di istruzione è riconosciuto in base al valore di alcuni bit (**7 bit**) meno significativi (**codice operativo – OPCODE**)

| 31                                                                                                                                      | 7 | 6 | 0                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------|---|---|---------------------------------------|
| immediato ( <i>imm</i> ) / indirizzi di registro ( <i>rd</i> , <i>rs1</i> , <i>rs2</i> )<br>codici operativi ausiliari ( <i>funct</i> ) |   |   | codice operativo<br>( <i>opcode</i> ) |

le istruzioni possono

- referenziare al massimo tre registri: due registri sorgente (*rs1* e *rs2*) e un registro destinazione (*rd*); tutti i registri sono a 64 bit
- indicare direttamente un valore numerico (*immediato*); il valore immediato ha significato diverso secondo il tipo di istruzione



# formato delle istruzioni RISC V – ii

le istruzioni RISC V sono di **sei** tipi (formati)

| formato                           | istruzioni                                                                                                      | immediato                          | immediato esteso (64 bit) |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------|---------------------------|
| <b>R</b> (register)               | aritmetico-logiche                                                                                              | na                                 | na                        |
| <b>I</b> (immediate)              | di caricamento da memoria (load)<br>aritmetico-logiche con immediato<br>salto incondizionato da registro & link | 12 bit in CPL2                     | estensione segno          |
| <b>S</b> (store)                  | di scaricamento in memoria (store)                                                                              | 12 bit in CPL2                     | estensione segno          |
| <b>B</b> (branch) (SB in PH)      | salto condizionato (branch)                                                                                     | 12 bit in CPL2,<br>multiplo di due | estensione segno          |
| <b>U</b> (load upper integer)     | istruzione <i>lui</i> (e <i>auipc</i> )                                                                         | 20 bit in CPL2                     | na                        |
| <b>J</b> (jump & link) (UJ in PH) | salto incondizionato (jump) &<br>collegamento (link)                                                            | 20 bit in CPL2,<br>multiplo di due | estensione segno          |

immediato:

- in *load* e *store* è un *offset* rispetto a un registro base per individuare l'indirizzo di memoria riferito al dato
- in *branch* e *jump* è un offset rispetto al Program Counter, o altro registro, per individuare l'indirizzo di memoria dell'istruzione destinazione del salto



# formato delle istruzioni RISC V – iii

## formato dell'istruzione a livello di linguaggio macchina

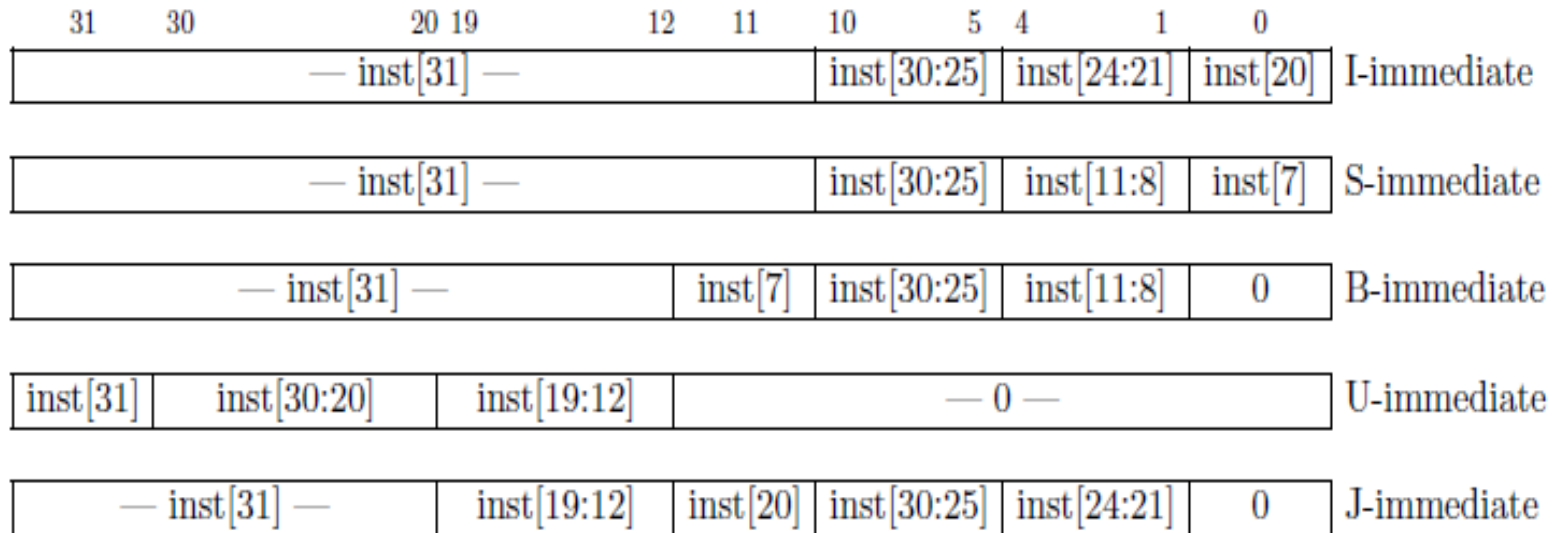
|            |    |           |    |     |     |         |     |            |        |        |          |          |        |         |        |        |  |        |
|------------|----|-----------|----|-----|-----|---------|-----|------------|--------|--------|----------|----------|--------|---------|--------|--------|--|--------|
| 31         | 30 | 25        | 24 | 21  | 20  | 19      | 15  | 14         | 12     | 11     | 8        | 7        | 6      | 0       |        |        |  |        |
| funct7     |    |           |    | rs2 |     |         | rs1 |            | funct3 |        | rd       |          | opcode |         | R-type |        |  |        |
| imm[11:0]  |    |           |    |     |     | rs1     |     | funct3     |        | rd     |          | opcode   |        | I-type  |        |        |  |        |
| imm[11:5]  |    |           |    | rs2 |     |         | rs1 |            | funct3 |        | imm[4:0] |          | opcode |         | S-type |        |  |        |
| imm[12]    |    | imm[10:5] |    |     | rs2 |         |     | rs1        |        | funct3 |          | imm[4:1] |        | imm[11] |        | opcode |  | B-type |
| imm[31:12] |    |           |    |     |     |         |     |            | rd     |        |          | opcode   |        |         | U-type |        |  |        |
| imm[20]    |    | imm[10:1] |    |     |     | imm[11] |     | imm[19:12] |        |        | rd       |          |        | opcode  |        | J-type |  |        |

- i numeri di registro *rs1*, *rs2* e *rd* (quando previsti) sono sempre nella stessa posizione all'interno dell'istruzione
- il campo *funct3* (e *funct7*) agisce come *estensione* del codice operativo per specificare esattamente che cosa fa l'istruzione, in aggiunta a quanto già indicato dal codice operativo
- il campo *funct3* (quando previsto) è sempre nella stessa posizione
- i bit dell'immediato (12 o 20) sono in posizioni diverse a seconda del formato



# formato delle istruzioni RISC V – iv

**costruzione dell'immediato** (su 32 bit) nei diversi formati



ANTICIPAZIONE SUL PROCESSORE: questa **costruzione degli immediati** viene fatta in **HW** da parte del modulo **ESTENSIONE DEL SEGNO** della pipeline del processore, *durante* la fase di decodifica del codice operativo, in modo da portare gli offset da 12 bit o 20 bit fino ad avere 64 bit, mediante estensione del segno (in CPL2); per i **formati B e J**, prima di sommare l'**offset** al PC, il processore **lo** deve riportare a byte (è riferito a mezza parola) **tramite moltiplicazione per due** (ossia tramite aggiunta di un bit 0 a dx)





# esempio

| Istruzione                           | Formato | funz7     | rs2 | rs1 | funz3 | rd        | codop   |
|--------------------------------------|---------|-----------|-----|-----|-------|-----------|---------|
| add                                  | R       | 0000000   | reg | reg | 000   | reg       | 0110011 |
| sub (sottrazione)                    | R       | 0100000   | reg | reg | 000   | reg       | 0110011 |
| Istruzione                           | Formato | immediato |     | rs1 | funz3 | rd        | codop   |
| addi (addizione immediata)           | I       | costante  |     | reg | 000   | reg       | 0010011 |
| ld (caricamento di parola doppia)    | I       | indirizzo |     | reg | 011   | reg       | 0000011 |
| Istruzione                           | Formato | immediato | rs2 | rs1 | funz3 | immediato | codop   |
| sd (memorizzazione di parola doppia) | S       | indirizzo | reg | reg | 011   | indirizzo | 0100011 |

codifica completa in linguaggio macchina di alcune istruzioni

| Istruzione (R)                       | funz7        | rs2   | rs1   | funz3 | rd        | codop   | Esempio          |
|--------------------------------------|--------------|-------|-------|-------|-----------|---------|------------------|
| add                                  | 0000000      | 00011 | 00010 | 000   | 00001     | 0110011 | add x1, x2, x3   |
| sub (sottrazione)                    | 0100000      | 00011 | 00010 | 000   | 00001     | 0110011 | sub x1, x2, x3   |
| Istruzione (I)                       | immediato    |       | rs1   | funz3 | rd        | codop   | Esempio          |
| addi (addizione immediata)           | 001111101000 |       | 00010 | 000   | 00001     | 0010011 | addi x1,x2,1000  |
| ld (caricamento di parola doppia)    | 001111101000 |       | 00010 | 011   | 00001     | 0000011 | ld x1, 1000 (x2) |
| Istruzione (S)                       | Immediato    | rs2   | rs1   | funz3 | immediato | codop   | Esempio          |
| sd (memorizzazione di parola doppia) | 0011111      | 00001 | 00010 | 011   | 01000     | 0100011 | sd x1, 1000 (x2) |



# corrispondenza tra formati e istruzioni macchina native

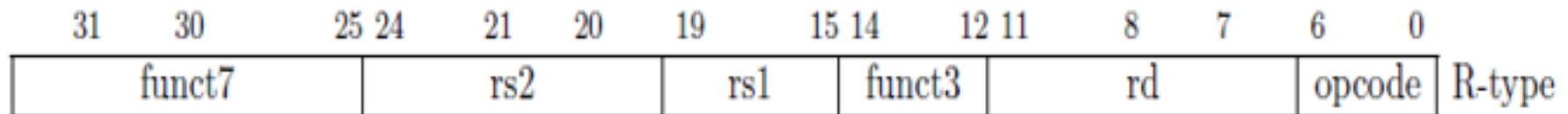
| formato  | istruzione                                                                                   |
|----------|----------------------------------------------------------------------------------------------|
| R        | <i>add, sub, or, and, xor, slt, sll, srl, sra</i>                                            |
| I        | <i>addi, ori, andi, xori, slti, slli, srli, srai</i><br><i>ld, lw, lh, lb</i><br><i>jalr</i> |
| S        | <i>sd, sw, sh, sb</i>                                                                        |
| B (o SB) | <i>beq, bne, bge, blt</i>                                                                    |
| U        | <i>lui, auipc</i>                                                                            |
| J (o UJ) | <i>jal</i>                                                                                   |



## formato istruzioni di tipo R – istruzioni aritmetico-logiche tra registri

|          |                                                   |
|----------|---------------------------------------------------|
| <b>R</b> | <i>add, sub, or, and, xor, slt, sll, srl, sra</i> |
|----------|---------------------------------------------------|

la rappresentazione binaria rispetta il seguente formato

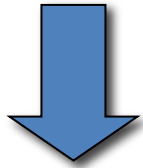


- **opcode**: (opcode) identifica il tipo di istruzione
- **rs1**: registro contenente il primo operando sorgente
- **rs2**: registro contenente il secondo operando sorgente
- **rd**: registro destinazione contenente il risultato
- **funct3**, **funct7**: campi che indicano la variante specifica dell'istruzione



# istruzioni di tipo R – esempio

add t0, s1, s2



| funct7 (7) | rs2 (5) | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|------------|---------|---------|------------|--------|------------|
| 0000000    | 10010   | 01001   | 000        | 00101  | 0110011    |

istruzione rappresentata in esadecimale

0x 0124 82B3



# istruzioni di tipo R – esempi

la combinazione dei campi *opcode*, *funct3* e *funct7* specifica esattamente l'operazione che il processore deve eseguire

`add s1, s2, s3`

| funct7 (7) | rs2 (5) | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|------------|---------|---------|------------|--------|------------|
| 0000000    | 10011   | 10010   | 000        | 01001  | 0110011    |

rappresentata in esadecimale

**0x 0139 04B3**

`sub s1, s2, s3`

| funct7 (7) | rs2 (5) | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|------------|---------|---------|------------|--------|------------|
| 0100000    | 10011   | 10010   | 000        | 01001  | 0110011    |

rappresentata in esadecimale

**0x 4139 04B3**



# operazioni logiche e istruzioni logiche

AND - utile per forzare bit specifici di una parola a 0

- nativa `and` tipo R o `andi` tipo I

OR - utile per forzare bit specifici di una parola a 1

- nativa `or` tipo R o `ori` tipo I

NOT – *pseudoistruzione* `not rd, rs1`

- realizzata come `xori rd, rs1, -1`      # `-1 = 0x F.....F`

ATTENZIONE: nel caso di formato I bisogna ricordare che il prolungamento del campo *imm12bit* fino a 64 bit avviene mediante estensione del segno

## esempio

voglio isolare i quattro bit più significativi di una parola contenuta nel registro *t2*: devo usare `and` tipo R, dopo avere inserita in un altro registro *t3* la costante `0x F000 0000 0000 0000` (vedremo come), ed eseguire un `and` bit-a-bit

```
and t2, t2, t3          # t2 = t2 & t3
```



# formato istruzioni di tipo I

I

*addi, ori, andi, xori, slti, slli, srli, srai*  
*ld, lw, lh, lb*  
*jalu*

la rappresentazione binaria rispetta il seguente formato



- **opcode**: (opcode) identifica il tipo di istruzione
- **rs1**: registro contenente il primo operando sorgente
- **rd**: registro destinazione contenente il risultato
- **funct3**: indica la variante specifica dell'istruzione
- **imm[11:0]**: valore dell'immediato su 12 bit in CPL2



# istruzioni di tipo I – esempi di istruzioni aritmetiche con operando immediato

`addi s1, s1, 4`

| imm [11:0]   | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|--------------|---------|------------|--------|------------|
| 000000000100 | 01001   | 000        | 01001  | 0010011    |

`slli s1, s0, 4`      # registro s0 << 4 bit

| imm [11:0]   | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|--------------|---------|------------|--------|------------|
| 000000000100 | 01000   | 001        | 01001  | 0010011    |

`slti t0, s2, 8`      # t0 = 1 if s2 < 8; t0 = 0 otherwise

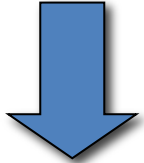
| imm [11:0]   | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|--------------|---------|------------|--------|------------|
| 000000001000 | 10010   | doc RISCv  | 00101  | doc RISCv  |





# istruzioni di tipo I – esempio ld

`ld t0, 32(s3)`



- **rs1**: registro base
- **rd**: registro destinazione contenente il risultato
- **imm[11:0]**: offset da sommare al registro base per ottenere l'indirizzo di memoria

| imm [11:0]   | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|--------------|---------|------------|--------|------------|
| 000000100000 | 10011   | 011        | 00101  | 0000011    |

rappresentata in esadecimale      **0x 0209 B283**

per esempio: con registro *s3* contenente la parola doppia **0x 0000 0000 1000 0010** e *offset12* = **0x 020** (ossia 32 in base dieci), viene letta e copiata in *t0* la parola doppia il cui byte meno significativo ha indirizzo **0x 0000 0000 1000 0030** (... RISC-V è *little endian*)



# formato istruzioni di tipo S

S

*sd, sw, sh, sb*

Non essendoci registri destinazione, ma solo registri sorgente la rappresentazione binaria rispetta il seguente formato

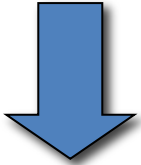
|           |     |     |        |          |        |        |
|-----------|-----|-----|--------|----------|--------|--------|
| imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode | S-type |
|-----------|-----|-----|--------|----------|--------|--------|

- **opcode**: (opcode) identifica il tipo di istruzione
- **rs1**: registro contenente il primo operando sorgente (registro base)
- **rs2**: registro contenente il secondo operando sorgente (dato da scrivere in memoria)
- **funct3**: indica la variante specifica dell'istruzione
- **imm[11:0]**: valore *ricostruito* dell'immediato su 12 bit in CPL2



# istruzioni di tipo S – esempio sd

**sd t0, 32(s3)**



- **rs1**: registro base
- **rs2**: registro contenente il valore da scrivere in memoria
- **imm[11:0]**: offset da sommare al registro base per ottenere l'indirizzo di memoria

| imm [11:5] | rs2 (5) | rs1 (5) | funct3 (3) | imm [4:0] | opcode (7) |
|------------|---------|---------|------------|-----------|------------|
| 0000001    | 00101   | 10011   | 011        | 00000     | 0100011    |

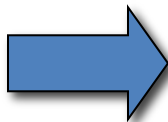
rappresentata in esadecimale

**0x 0259 B023**



# esempio ld e sd

$A[150] = h + A[150]$



con **h** in **s2** e indirizzo base **A** in **t1**

```
ld    t0, 1200(t1)
add   t0, s2, t0
sd    t0, 1200(t1)
```

| imm [11:0]   | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|--------------|---------|------------|--------|------------|
| 010010110000 | 00110   | 011        | 00101  | 0000011    |

| funct7 (7) | rs2 (5) | rs1 (5) | funct3 (3) | rd (5) | opcode (7) |
|------------|---------|---------|------------|--------|------------|
| 0000000    | 00101   | 10010   | 000        | 00101  | 0110011    |

| imm [11:5] | rs2 (5) | rs1 (5) | funct3 (3) | imm [4:0] | opcode (7) |
|------------|---------|---------|------------|-----------|------------|
| 0100101    | 00101   | 00110   | 011        | 10000     | 0100011    |



# istruzioni di salto

salto condizionato relativo a PC:

- **beq** *rs1*, *rs2*, *L1*  
(*branch on equal*)
- **bne** *rs1*, *rs2*, *L1*  
(*branch on not equal*)
- e anche **bge** e **blt**

|                                      | formato  |
|--------------------------------------|----------|
| <i>beq, bne, bge, blt</i>            | <b>B</b> |
| jump & link: <i>jal</i>              | <b>J</b> |
| jump & link da registro: <i>jalr</i> | <b>I</b> |

salto incondizionato con collegamento:

- **jal** *rd*, *L1* (*jump & link*)
- **jalr** *rd*, *spi12(rs1)*  
(*jump da registro & link*)

con *collegamento* significa che in *rd* viene salvato l'indirizzo dell'istruzione successiva a quella di salto



# istruzioni di salto

salto condizionato:

- il flusso sequenziale di controllo cambia solo se la condizione è vera
- il calcolo del valore dell'etichetta **11** (indirizzo di destinazione del salto) è relativo al Program Counter (PC) dell'istruzione di salto

salto incondizionato con collegamento:

- il salto viene sempre eseguito
- l'indirizzo dell'istruzione successiva a quella di salto ( $PC + 4$ ) è memorizzato nel registro *rd*
- il calcolo dell'indirizzo di destinazione del salto è
  - **jal**: relativo al Program Counter (PC) dell'istruzione di salto
  - **jalr**: relativo al registro *rs1*, specificato come primo operando sorgente

il calcolo dell'indirizzo destinazione di salto è **relativo** (a PC o *rs1*)

- identificato da uno **spiazzamento in CPL2**: 12 bit per i formati B e I, e 20 bit per il formato J

l'indirizzo di un'istruzione è a parola, ossia è sempre multiplo di 4

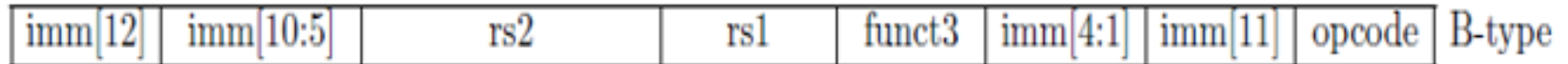
- **convenzioni e operazioni HW apposite assicurano che la somma di registro base e spiazzamento produca sempre un indirizzo di istruzione che sia multiplo di 4**



# formato istruzioni di tipo B – i

|                 |                           |
|-----------------|---------------------------|
| <b>B (o SB)</b> | <i>beq, bne, bge, blt</i> |
|-----------------|---------------------------|

la rappresentazione binaria rispetta il seguente formato



- **opcode**: (opcode) identifica il tipo di istruzione
- **rs1**: primo operando sorgente
- **rs2**: secondo operando sorgente (dato da scrivere in memoria)
- **funct3**: indicano la variante specifica dell'istruzione per effettuare il confronto tra *rs1* e *rs2*
- **imm[12:1]**: valore *ricostruito* dello spiazzamento su 12 bit in CPL2



# formato istruzioni di tipo B – ii

1. Lo spiazzamento (rispetto al PC) ha a disposizione solo **12 bit usati intendendo ogni unità come una mezza parola**.
2. Poiché gli **indirizzi di istruzioni sono al byte** e sono numericamente un **multiplo di 4**, il numero di mezze parole da sommare a PC dovrà per forza essere un numero pari (multiplo di due).
3. Per non «sprecare» il bit meno significativo, che è sempre nullo,
  - a) nel passare da linguaggio assembly a linguaggio macchina, la stringa in linguaggio macchina non lo include.
  - b) Nell'eseguire l'istruzione in linguaggio macchina, la CPU
    - i. riordina i pezzetti dei 12 bit di immediato presenti nella stringa in formato B
    - ii. aggiunge uno zero in coda (il LSB sottointeso perché lo spiazzamento in mezze parole è pari)
    - iii. estende il segno della costante così ottenuta
    - iv. effettua moltiplicazione per due della costante (per convertire da mezze parole a byte)
    - v. Somma la costante (codificata ora su 64 bit) al PC





# istruzioni di salto condizionato – tipo B

l'indirizzamento relativo al Program Counter permette di fare salti condizionati ad aree di memoria il cui indirizzo non è esprimibile con solo 12 bit

esempio: **bne s0, s1, L1**

- per esprimere l'etichetta **L1** si dispone solo di 12 bit
- $\Rightarrow$  il valore di **L1** è calcolato rispetto al Program Counter in modo da saltare di  $+2^{12}-1$  mezze parole avanti o  $-2^{12}$  indietro rispetto all'istruzione corrente

per il **principio di località** degli indirizzi di memoria è utile calcolare l'indirizzo di destinazione del salto come **offset** rispetto all'istruzione corrente



# istruzioni di salto condizionato – tipo B

i 12-bit del campo **indirizzo** esprimono lo **offset** rispetto al PC rappresentato in complemento a due, per permettere salti in avanti e indietro

Il salto varia (vedi slide precedente) tra  $-2^{12}$  e  $+2^{12}-1$  mezze parole  
esempio: **bne s0, s1, L1**

l'assemblatore sostituisce l'etichetta **L1** con lo spiazzamento alla **mezza parola** relativo a PC:  $(L1 - PC) / 2$ , dove

- PC contiene l'indirizzo dell'istruzione di salto
- la divisione per due serve per calcolare la distanza in mezze parole

il valore del campo **indirizzo** può anche essere negativo (salto all'indietro)



# istruzioni di tipo B – esempio

**beq s1, s2, L1**

dove:

**L1 = indirizzo a 64bit del byte iniziale  
dell'istruzione che è alla destinazione del salto**

**PC = indirizzo del byte iniziale dell'istruzione di  
salto condizionato**

**$50_{\text{dec}} = (L1 - PC) / 2 = \text{offset di mezza parola}$**

nell'esempio la distanza in byte tra  
l'istruzione destinazione del salto  
e l'istruzione di salto condizionato  
è pari a  $100_{\text{dec}}$

$100_{\text{dec}}$  in binario **1100100**

| imm [12][10:5]   | rs2 (5)      | rs1 (5)      | funct3 (3)       | imm [4:1] [11] | opcode (7)       |
|------------------|--------------|--------------|------------------|----------------|------------------|
| <b>0 0000011</b> | <b>10010</b> | <b>01001</b> | <b>doc RISCv</b> | <b>1001 0</b>  | <b>doc RISCv</b> |

**$50_{\text{dec}} = 0000000110010$**  *ci sono a disposizione 13 bit  
in cpl2, perché l'LSB che è sempre nullo non si  
materializza nel formato binario, lasciando 12 bit in  
cpl2 per un conteggio di mezza parole*

**000000011001**<sub>12bit\_cpl2</sub> ==  
**000000011001**



# esempio

|       |       |              |                        |       |       |
|-------|-------|--------------|------------------------|-------|-------|
| LOOP: | add   | t1, s3, s3   | 80000 <sub>dec</sub> : | add   | ...   |
|       | ..... |              |                        | ..... |       |
|       | bne   | t0, s5, EXIT | 80016 <sub>dec</sub> : | bne   | ... 6 |
|       | add   | s3, s3, s4   | 80020 <sub>dec</sub> : | ...   |       |
|       | j     | LOOP         | 80024 <sub>dec</sub> : | ...   |       |
| EXIT: | ..... |              | 80028 <sub>dec</sub> : | EXIT  | ..... |

Nota: *j LOOP* *pseudoistruzione* equivalente a *jal zero, LOOP*

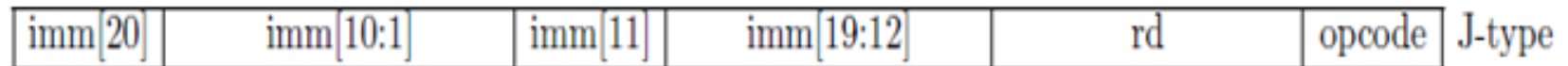
$$6_{\text{dec}} = (80028_{\text{dec}} - 80016_{\text{dec}}) / 2$$



# salto incondizionato con collegamento – formato istruzioni di tipo J

**J (o UJ)** | *jal rd, imm20*

la rappresentazione binaria rispetta il seguente formato



- **opcode**: (opcode) identifica il tipo di istruzione
- **rd**: registro destinazione per il salvataggio di PC + 4
- **imm[20:1]**: valore «ricostruito» dello spiazzamento su 20 bit in CPL2

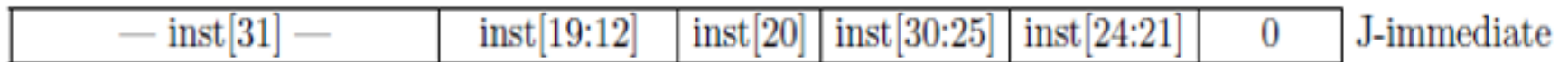


# formato istruzioni di tipo J

lo spiazzamento (rispetto al PC) ha a disposizione solo 20 bit

lo spiazzamento **deve** fare riferimento ad *half word* (mezza parola), pertanto esso termina sempre con un bit 0 che viene sottointeso nella rappresentazione binaria, consentendo di raddoppiare lo spazio di offset

il modulo HW **ESTENSIONE DEL SEGNO**, durante la fase di decodifica del codice operativo, ri-costruisce l'offset da 20 bit, lo prolunga fino a 64 bit mediante estensione del segno e **aggiunge il bit 0 finale (meno significativo) sottointeso** nella rappresentazione binaria dell'istruzione (nella figura si vede estensione a 32 bit)



pertanto, prima di sommare l'offset al PC, il processore riporta a byte l'offset (già esteso in segno) **tramite moltiplicazione per due** (scorrimento a sx di una posizione e aggiunta di un bit 0 a dx)



## salto incondizionato da registro con collegamento – formato istruzioni di tipo I – i

I     *jalr rd, imm12(rs1)*

la rappresentazione binaria rispetta il seguente formato



- **opcode**: (opcode) identifica il tipo di istruzione
- **rs1**: registro contenente la base di salto
- **rd**: registro destinazione per il salvataggio di PC + 4
- **funct3**: indica la variante specifica dell'istruzione
- **imm[11:0]**: valore dell'offset su 12 bit in CPL2



## salto incondizionato da registro con collegamento – formato istruzioni di tipo I – ii

l'istruzione **jalr** prevede per il salto l'indicazione di uno spiazzamento a 12 bit (conteggio al byte) da sommare al valore di un registro a 64 bit. Nel formato binario I non viene sottointeso nessun bit.

Dovendo valere il vincolo che l'indirizzo indicato nell'istruzione sia almeno un multiplo di 2 per compatibilità con l'estensione «C» dell'ISA RV64ic (che descrive il formato delle istruzioni compresse codificate in binario su 16-bit) lo spiazzamento viene inteso come un numero pari (LSB nullo) e se non lo è il suo LSB è forzato a zero.





# pseudoistruzioni di salto incondizionato

|                  |                                      |                                                                                                                                                           |                                                                 |
|------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>j spi20</b>   | $pc := pc + spi20$                   | salto incondizionato – salto relativo a PC                                                                                                                | <b>jal zero, spi20</b><br>// non salva ind. di rientro          |
| <b>jr rs1</b>    | $pc := rs1$                          | salto incondizionato a lunga distanza – salto da registro <i>rs1</i>                                                                                      | <b>jalr zero, 0(rs1)</b><br>// non salva ind. di rientro        |
| <b>jal spi20</b> | $ra := pc + 4$<br>$pc := pc + spi20$ | chiamata a sottoprogramma – salto incondizionato relativo a PC con salvataggio dell'indirizzo di rientro nel registro <i>ra</i>                           | <b>jal ra, spi20</b><br>// salva in <i>ra</i> ind. di rientro   |
| <b>jalr rs1</b>  | $ra := pc + 4$<br>$pc := rs1$        | chiamata a sottoprogramma a lunga distanza – salto incondizionato da registro <i>rs1</i> con salvataggio dell'indirizzo di rientro nel registro <i>ra</i> | <b>jalr ra, 0(rs1)</b><br>// salva in <i>ra</i> ind. di rientro |
| <b>ret</b>       | $pc := ra$                           | rientro da sottoprogramma – salto da registro <i>ra</i>                                                                                                   | <b>jalr zero, 0(ra)</b><br>// non salva ind. di rientro         |



# gestione delle costanti su 32 bit

le istruzioni di **tipo I** consentono di rappresentare costanti esprimibili su 12 bit in CLP2, nell'intervallo chiuso  $[-2048, +2047]$

per avere una costante (o anche un indirizzo) a **32 bit** è necessario *caricarla* nel registro in due passi (tramite due istruzioni), suddividendo il valore della costante in due parti, una da 20 bit e l'altra da 12 bit, le quali vengono trattate separatamente come due *immediati* tramite due istruzioni successive:

- si utilizza l'istruzione **lui** (*load upper immediate*) per caricare il *primo immediato* (che rappresenta i 20 bit più significativi della costante) nei 20 bit più significativi della metà inferiore da 32 bit di un registro; i 12 bit meno significativi della metà inferiore del registro vengono azzerati e il segno della metà inferiore del registro a 64 bit viene esteso fino al 64-esimo bit
- una successiva istruzione **addi** consente di aggiungere, tramite il *secondo immediato*, i 12 bit meno significativi della costante



# formato istruzioni di tipo U

|          |                   |
|----------|-------------------|
| <b>U</b> | <i>lui, auipc</i> |
|----------|-------------------|

**lui**          load upper immediate

**auipc**        add upper immediate to PC

la rappresentazione binaria rispetta il seguente formato



- **opcode**: (opcode) identifica il tipo di istruzione
- **rd**: registro destinazione del valore costante o indirizzo
- **imm[31:12]**: valore della costante / indirizzo su 20 bit in CPL2

valore dell'immediato a 32 bit



# istruzioni di tipo U

|                              |                                                |                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>lui rd, cost20</code>  | $rd := \text{cost20}$<br>(solo alcuni bit)     | caricamento (parziale) di costante – <ul style="list-style-type: none"><li>• carica la costante <i>cost20</i> nei 20 bit (5 cifre hex) più significativi della metà inferiore da 32 bit (8 cifre hex) del registro <i>rd</i></li><li>• azzera i 12 bit (3 cifre hex) meno significativi di <i>rd</i></li><li>• estende in segno la metà inferiore di <i>rd</i> fino al 64-esimo bit</li></ul> |
| <code>auipc rd, spi20</code> | $rd := pc + \text{spi20}$<br>(solo alcuni bit) | caricamento (parziale) di indirizzo relativo a PC – <ul style="list-style-type: none"><li>• prima somma lo spiazzamento <i>spi20</i> all'indirizzo dell'istruzione <i>auipc</i> (contenuto nel registro PC)</li><li>• poi lo tratta come la costante nell'istruzione <i>lui</i> e lo carica nel registro <i>rd</i></li></ul>                                                                  |

di solito la costante *cost20* è specificata come valore numerico o come costante simbolica dichiarata tramite la direttiva **.eqv**

lo spiazzamento *spi20* esprime la distanza (in byte) e di solito è specificato come etichetta di istruzione o di dato

le istruzioni **lui** e **auipc** sono usate per espandere le pseudo-istruzioni **li** e **la**, rispettivamente



# ancora sulla gestione delle costanti su 32 bit – i

```
.eqv const32 0x7E1DF877  
li rd, const32
```

costante a 32 bit da caricare nei 32 bit meno significativi di un registro di destinazione (rd è a 64 bit)

$0x7E1DF877 == 0b\ 0111111100001110111111100001110111$

la costante va interpretata come

$0x\ 7E1DF = 0b\ 0111111100001110111111 = \text{immediato\_20bit}$  da caricare tramite **lui**

$0x\ 877 = 0b\ 100001110111 = \text{immediato\_12bit}$  da caricare tramite **addi**

```
li rd, const 32 # viene espansa dall'assemblatore in  
lui rd, 0x7E1DF  
addi rd, rd, 0x877
```

gli immediati vengono trattati in CLP2, pertanto **immediato\_12bit** *risulta* negativo (MSB di  $100001110111 == 1$ ) e l'istruzione **addi** lo somma dopo averne esteso il *segno*:

$00000000\ 7E1DF000 +$   
 $FFFFFFFF\ FFFFFFF877 =$   
 $00000000\ 7D1DE877$  **che è errato!**



# ancora sulla gestione delle costanti su 32 bit – i

```
.eqv const32 0x7E1DF877  
li rd, const32
```

costante a 32 bit da caricare nei 32 bit meno significativi di un registro di destinazione (rd è a 64 bit)

$0x7E1DF877 == 0b\ 0111111100001110111111100001110111$

la costante va interpretata come

$0x\ 7E1DF = 0b\ 0111111100001110111111 = \text{immediato\_20bit}$  da caricare tramite **lui**

$0x\ 877 = 0b\ 100001110111 = \text{immediato\_12bit}$  da caricare tramite **addi**

```
li rd, const 32 # viene espansa dall'assemblatore in  
lui rd, 0x7E1DF  
addi rd, rd, 0x877
```

il caricamento della costante a 32 bit nel registro avviene in modo corretto se i 20 bit che costituiscono l'immediato di **lui** sono costruiti come

$\text{immediato\_20bit}[31:12] + \text{immediato\_12bit}[11]$

Esecuzione della **lui** mette in **rd** :

$00000000(7E1DF+1)000 == 000000007E1E0000$

Dopo esecuzione della **addi** in **rd** troviamo il valore corretto:

$00000000\ 7E1E0000+$

$FFFFFFFF\ FFFFF877 =$

$00000000\ 7E1DF877$  che è quello che si voleva con «**li rd, const32**»



# ancora sulla gestione delle costanti su 32 bit – i

## RIEPILOGO

`lui rd, 0x7E1DF877` # in isolamento, viene eseguita dall'assemblatore mettendo in rd  
`00000000 7E1DF000`

`li rd, 0x7E1DF877` # viene espansa dall'assemblatore come  
`lui rd, %hi(7E1DF877)`  
`addi rd, rd, %lo(7E1DF877)`

La scrittura `lui rd, %hi(7E1DF877)` # viene eseguita dall'assemblatore mettendo in rd  
`00000000(7E1DF + bit più significativo di 877)000 == 000000007E1E0000`

La scrittura `addi rd, rd, %lo(7E1DF877)` # viene eseguita dall'assemblatore sommando in rd  
`FFFF FFFF FFFF F877` # si estende il segno di 877 a riempire 64bit



# pseudoistruzioni di caricamento in registro di costante e indirizzo a 32 bit – i

|                            |                           |                                                   |
|----------------------------|---------------------------|---------------------------------------------------|
| <code>li rd, cost32</code> | $rd := \text{cost32}$     | caricamento (completo) di costante                |
| <code>la rd, spi32</code>  | $rd := pc + \text{spi32}$ | caricamento (completo) di indirizzo relativo a PC |

`cost32` / `spi32` possono essere direttamente un **valore / indirizzo** a 32 bit oppure una **costante simbolica** / un'**etichetta di istruzione o dato**

l'assemblatore (e il collegatore – *linker*) espande la pseudo-istruzione *li* / *la* nelle istruzioni native *lui* / *auipc* seguite dall'istruzione *addi*, suddividendo il valore numerico da 32 bit nelle due parti: 20 bit superiori (**hi**) e 12 bit inferiori (**lo**)

per rendere i valori coerenti con il funzionamento di *lui* / *auipc* seguite da *addi*, l'assemblatore *rappresenta* **hi** e **lo** utilizzando un *modificatore*, e il calcolo relativo viene svolto dal collegatore





# sulla gestione delle costanti per *load address (la)*

```
.eqv indMem 0x000000007E1DF877
```

```
Label: la rd, indMem
```

Questa pseudo istruzione viene espansa dall'assemblatore in

```
Label: auipc rd, %hi_pcrel(indMem)
      addi rd, rd, %lo_pcrel(indMem)
```

In fase di esecuzione **%hi\_pcrel(indMem)** indica le operazioni:

1. Calcolo di `indMem-Label` (è responsabilità del programmatore che il risultato della sottrazione sia correttamente rappresentato su 32 bit)
2. I 20 bit superiori di `indMem-Label` sommati con MSB dei 12 bit meno significativi di `indMem-Label` vanno a sostituire i 20 bit più significativi della parola inferiore di `rd`;
3. i restanti bit della parola inferiore di `rd` sono posti a zero;
4. la parola più significativa di `rd` contiene l'estensione del segno della meno significativa.

In fase di esecuzione **%lo\_pcrel(indMem)** indica che l'immediato da sommare sono:

1. i 12 bit meno significativi di `indMem-Label`

```
auipc rd, indMem # in isolamento, crea i 32bit da mettere in rd come i 20 bit più significativi
                  # di indMem-PC seguiti da 12 bit a zero, ed estende il segno nella parola più
                  # significativa
```



# pseudoistruzioni di caricamento in registro di costante e indirizzo – ii

| modificatore                                                                    | calcolo del modificatore – da parte del linker |                        |
|---------------------------------------------------------------------------------|------------------------------------------------|------------------------|
| <code>%hi(val)</code>                                                           | <code>= val[31:12] + val[11]</code>            | // 20 bit sup corretti |
| <code>%lo(val)</code>                                                           | <code>= val[11:0]</code>                       | // 12 bit inf          |
| si ponga: <code>delta = ind - pc</code> // l'indirizzo viene relativizzato a PC |                                                |                        |
| <code>%pcrel_hi(ind)</code>                                                     | <code>= delta[31:12] + delta[11]</code>        | // 20 bit sup corretti |
| <code>%pcrel_lo(ind)</code>                                                     | <code>= delta[11:0]</code>                     | // 12 bit inf          |

| pseudo-istruzione | espansione – in istruzioni native e modificatori |                          |                    |
|-------------------|--------------------------------------------------|--------------------------|--------------------|
| li rd, valore     | lui                                              | rd, %hi(valore)          | // 20 bit sup - HI |
|                   | addi                                             | rd, rd, %lo(valore)      | // 12 bit inf - LO |
| la rd, indirizzo  | auipc                                            | rd, %pcrel_hi(indir)     | // 20 bit sup - HI |
|                   | addi                                             | rd, rd, %pcrel_lo(indir) | // 12 bit inf - LO |



# modalità di indirizzamento – i

le **modalità di indirizzamento** indicano sintatticamente quale valore considerare come operandi delle istruzioni in linguaggio assembler -- gli operandi di una istruzione possono essere valori nei registri, valori in memoria o valori numerici specificati direttamente (immediati)

l'esempio più comune di modalità di indirizzamento è l'indirizzamento **a registro**, dove gli operandi dell'istruzione sono contenuti nei registri:

esempio: `add s0, s1, s2`



# modalità di indirizzamento – ii

RISC V ha solo quattro modalità di indirizzamento:

- a registro
- immediato
- con base e spiazzamento
- relativo al Program Counter (PC)

una singola istruzione può fare uso di due o più modalità di indirizzamento distinte (come p.es. in **addi**)

Nota: altri processori (per esempio Intel) hanno modalità di indirizzamento più numerose (ma non molte di più di RISC V).



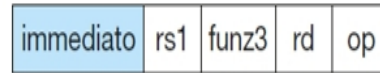
# modi di indirizzamento in RISC V

## immediato:

**costanti & registri**

**addi**

### 1. Indirizzamento immediato

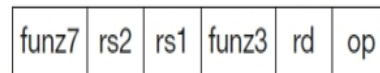


## registro:

**solo registri**

**add**

### 2. Indirizzamento tramite registro



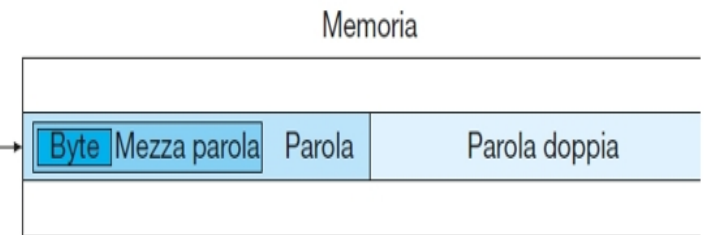
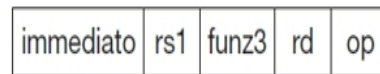
## base e spiazzamento:

**memoria[registro + costante]**

**ld, sd**

**jalr**

### 3. Indirizzamento tramite base



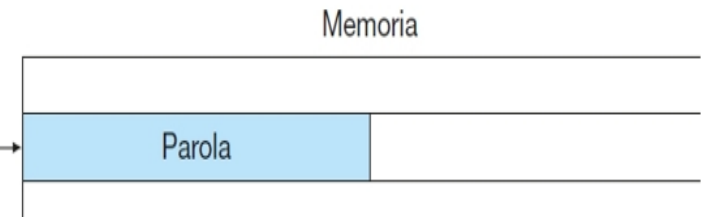
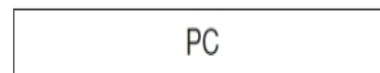
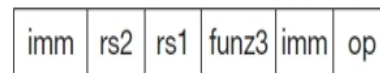
## PC-relativo:

**PC + costante**

**beq**

**jal**

### 4. Indirizzamento relativo al PC



## indirizzamento a registro

l'operando (l'indirizzo) è il contenuto di un registro della CPU: il nome del registro è specificato nell'istruzione

## indirizzamento immediato

l'operando è una costante il cui valore è contenuto nell'istruzione

l'indirizzamento immediato serve per specificare il valore di un operando sorgente; non ha senso usarlo come destinazione



# indirizzamento con base

l'operando è in una locazione di memoria il cui indirizzo si ottiene sommando il contenuto di un registro base e il valore costante (*offset* o *spiazzamento*) contenuto nell'istruzione

le istruzioni che usano questo tipo di indirizzamento hanno formato di tipo I (*ld* e *jalr*) e di tipo S (*sd*)

l'offset è di 12 bit: può quindi assumere un valore tra  $-2^{11}$  e  $+2^{11} - 1$

# indirizzamento relativo a PC

l'operando si trova in una locazione di memoria il cui indirizzo si ottiene sommando il contenuto del *Program Counter* e il valore costante (*offset* o *spiazzamento*) contenuto nell'istruzione

le istruzioni che usano questo tipo di indirizzamento hanno formato

- di tipo B (salto condizionato) con offset ad *half word* su 12 bit
- di tipo J (salto incondizionato con collegamento) con offset ad *half word* su 20 bit



# utilizzo di costanti

spesso le operazioni richiedono l'uso di costanti

esempio: sommare a un registro il valore decimale 4

possibili opzioni:

- la costante risiede in memoria e viene caricata con `ld`
- utilizzare registri speciali (p.es. il registro `zero`)
- utilizzare **istruzioni di tipo I** in cui un operando è una **costante** (12 bit, funziona solo con costanti piccole ...)
- la costante a 32 bit (ma anche di dimensioni maggiori ...) viene (pre)caricata in un registro





# esempi

```
addi s0, s0, 4      # s0 ← s0 + 4 (sign-extended)
slti t0, s2, 10     # t0 = 1 if s2 < 10
andi s0, s0, 6      # s0 ← s0 and 6 (sign-extended)
ori  s0, s0, 10     # s0 ← s0 or 10 (sign-extended)
li   s0, 20         # s0 ← 20 (pseudo-instruction)
```

i valori immediati sono espressi in assembler in rappresentazione decimale, ma possono anche essere espressi come valori esadecimali o binari, preponendo loro i suffissi appositi



# esempio di indirizzamento a registri & immediato – i

```
# somma
```

```
.text                # definizione del segmento codice  
.globl main
```

```
main:
```

```
    li    t1, 10      # carica in t1 il valore decimale 10  
    li    t2, 15      # carica in t2 il valore decimale 15  
    add   a2, t2, t1   #  $a2 \leftarrow t2 + t1$ 
```

```
print_result:
```

```
    mv    a0, a2       # risultato da stampare (preparato in a0)  
    li    a7, 1        # numero del servizio richiesto (1 = print)  
    ecall                # chiama il SO per eseguire servizio print
```

Nota: l'istruzione speciale *ecall* di RISC V invoca il sistema operativo per svolgere un servizio di SO (qui *print*); essa presuppone di avere caricato in registri predefiniti l'argomento da stampare e il numero del servizio; come esempio di servizi semplici, si veda il simulatore RARS (che usa i registri *a0* e *a7*); il SO Linux ha oltre 300 servizi e un passaggio parametri più complesso.



## esempio di indirizzamento a registri & immediato – ii

```
# l'accesso immediato è usato anche dalle operazioni
# aritmetiche

    .text                # definizione del segmento codice
    .globl main

main:
    li    t1, 10          # t1 ← 10
    addi  a2, t1, 15       # a2 ← t1 + 15

# stampa risultato
print_result:
    move  a0, a2
    li    a7, 1
    ecall
```



## esempio di indirizzamento a registri & immediato – iii

```
# i valori immediati possono essere dichiarati come  
# esadecimali: 10 e 15 espressi in esadecimale con  
# 0x A e 0x F
```

```
    .text  
    .globl main  
main:  
    li    t1, 0x A  
    addi a2, t1, 0x F  
  
# stampa risultato  
print_result:  
    move a0, a2  
    li    a7, 1  
    ecall
```

